Machine Learning Course Notes

January 20, 2026

Contents

1 Week 1: Decision Theory

1.1 Introduction

In these notes, we briefly cover the basic notions of decision theory. The setting follows: there is a joint distribution D over $\mathcal{X} \times \mathcal{Y}$, where $\mathcal{X}$ is the input or feature space and $\mathcal{Y}$ is the label space. We aim to find a good predictor, i.e., a function $f : \mathcal{X} \to \mathcal{Y}$ that maps feature vectors to the corresponding label $y \in \mathcal{Y}$. To mathematically measure the performance of any such predictor, we need to specify a performance metric. We then assume to have a loss function $l : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$. In practice, $l(a, b)$ measures the loss when the predictor guesses label a, while the actual label is b.

Example 1.1. We report here the most common loss functions:

- The 0-1 loss, common for classification: $l(a, b) = \mathbb{1}\{a \neq b\}$
- The squared loss for regression: $l(a, b) = (a - b)^2$
- The absolute loss for regression: $l(a, b) = |a - b|$

Example 1.2 (Binary Classification). Consider a binary classification task, i.e., when $\mathcal{Y} = \{0, 1\}$. Any loss for this problem is fully specified by four numbers: $l(1, 1)$ (the loss value for true positive TP), $l(0, 0)$ (for true negative TN), $l(1, 0)$ (for false positive), and $l(0, 1)$ (for false negative).

While the distribution D is given by the environment, the loss function is a modeling choice of the model designer. Depending on the application at hand, you may want to choose specific loss functions.

Definition 1.3 (Expected Risk). Given distribution D on $\mathcal{X} \times \mathcal{Y}$ and loss function $l : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$, the expected risk of a predictor $f : \mathcal{X} \to \mathcal{Y}$ is defined as $R(f) = \mathbb{E}[l(f(X), Y)]$.

We remark that the expectation in the definition of risk is with respect to the random sampling of (X, Y) according to D.

Example 1.4. Consider a binary classification task on $\mathcal{X} = \{1, 2, 3\}$, with the 0-1 loss, and a predictor f that maps $\{1, 3\}$ in 1 and $\{2\}$ in 0. We want to compute its expected risk when (X, Y) follows the following distribution:

$$\begin{aligned} &\mathbb{P}(X = 1, Y = 0) = 0, && \mathbb{P}(X = 1, Y = 1) = 1/6. \\ &\mathbb{P}(X = 2, Y = 0) = 1/2, && \mathbb{P}(X = 2, Y = 1) = 1/12, \\ &\mathbb{P}(X = 3, Y = 0) = 1/12, && \mathbb{P}(X = 3, Y = 1) = 1/6 \end{aligned}$$

We have the following calculation: $R(f) = \frac{1}{12} \cdot 1 + \frac{1}{12} \cdot 1 = \frac{1}{6}$.

It is then natural to conclude that the best predictor given a fixed distribution D and loss function l is one that minimizes the expected risk.

Definition 1.5 ((Bayes) Optimal Predictor). Given distribution D on $\mathcal{X} \times \mathcal{Y}$ and loss function $l : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$, a predictor $f : \mathcal{X} \to \mathcal{Y}$ is said to be optimal if it minimizes the expected risk $R(f)$.

Exploiting the rules of conditional expectation, it is fairly easy to derive a closed formula for the optimal predictor: it maps features to the label that minimizes the loss, given the realized feature.

Theorem 1.6. The optimal predictor f^* is defined as follows.

$$f^*(x) = \arg\min_{z \in \mathcal{Y}} \mathbb{E}[l(z, Y) \mid X = x] \quad \forall x \in \mathcal{X}.$$

Note: the expectation in the definition is only with respect to Y, and it is conditioned with respect to $X = x$. Furthermore, the optimal predictor may not be unique, as the arg min could contain more than one element for some x.

Proof. We start by writing down explicitly the expected risk for a generic predictor f:

$$R(f) = \mathbb{E}[l(f(X), Y)] = \mathbb{E}[\mathbb{E}[l(f(X), Y) \mid X]].$$

The predictor f^* minimizes the term $\mathbb{E}[l(f(X), Y) \mid X]$ for each $x \in \mathcal{X}$, therefore it is optimal. □

The expression above may seem mysterious, but it is fairly simple if applied to specific distributions. For instance, if Y is discrete, then it can be rewritten as:

$$f^*(x) = \arg\min_{z \in \mathcal{V}} \sum_{y \in \mathcal{V}} l(z, y) \mathbb{P}(Y = y \mid X = x).$$

Similarly, if Y conditioned on $X = x$ admits a probability density function $p_{Y|X=x}$ for any $x \in \mathcal{X}$, then the optimal predictor is

$$f^*(x) = \arg\min_{z \in \mathcal{Y}} \int_{\mathcal{Y}} l(z, y) p_{Y|X=x}(y) dy.$$

Example 1.7. The predictor studied in Exercise 1.4 is optimal for that problem.

Important:

- Computing the Bayes optimal predictor requires exact knowledge of D on $\mathcal{X} \times \mathcal{Y}$.

- There is an inherent risk that is unavoidable! There is generally no perfect predictor.

Exercise 1.8. Consider a regression problem on $\mathcal{X} \times \mathcal{Y}$, with $\mathcal{Y} = \mathbb{R}$, and the quadratic loss defined as $l(a, b) = (a - b)^2$. Compute the optimal predictor.

Exercise 1.9. Consider a regression problem on $\mathcal{X} \times \mathcal{Y}$, with $\mathcal{Y} = \mathbb{R}$, and the absolute loss defined as $l(a, b) = |a - b|$. Compute the optimal predictor. Hint: Assume that Y conditioned on $X = x$ always admits a density function.

Exercise 1.10. Consider a classification task on k classes with the 0-1 loss. Compute the expected risk of the randomized predictor that outputs a class uniformly at random, independently of the x observed.

Exercise 1.11. Consider the binary classification problem with $\mathcal{Y} = \{-1, 1\}$ and the 0-1 loss. Relate the risk of a predictor f to that of its opposite -f.

Exercise 1.12 (Multi-Variate Independent Gaussians). Consider a multi-class classification problem where $\mathcal{X} = \mathbb{R}^d$, $\mathcal{Y} = \{1, 2, \ldots, k\}$, and the loss is the 0-1 loss. We make an assumption on the random variable $X = (X_1, \ldots, X_d)$: conditioning on any $Y = j$, X_i is an independent gaussian distribution with mean μ_i^j and standard deviation σ_i^j. Compute the optimal predictor.

1.2 Binary Classification

In binary classification, the label set $\mathcal{Y}$ comprises only two outcomes: true/false, accept/reject, positive/negative. We use $\mathcal{Y} = \{0, 1\}$ for simplicity. Any loss function for a binary classification problem is thus characterized by four numbers, corresponding to the elements of $\{0, 1\}^2$.

Proposition 2.1. In binary classification with loss l, the optimal predictor is given by

$$f^*(x) = \mathbb{1}\left\{\mathbb{P}(Y = 1 \mid X = x) \geq \frac{l(0,1) - l(0,0)}{l(1,0) - l(1,1)}\mathbb{P}(Y = 0 \mid X = x)\right\}$$

Proof. Consider any fixed $x \in \mathcal{X}$, we want to understand what the best outcome to predict is. We can compute explicitly the expected loss for the two options we have:

$$\begin{cases} l(1,1)\mathbb{P}(Y = 1 \mid X = x) + l(1,0)\mathbb{P}(Y = 0 \mid X = x) & \text{if we predict 1} \\ l(0,1)\mathbb{P}(Y = 1 \mid X = x) + l(0,0)\mathbb{P}(Y = 0 \mid X = x) & \text{if we predict 0} \end{cases}$$

Therefore, it is optimal to predict 1 if the first equation is smaller than or equal to the second one, and 0 otherwise. The statement of the Proposition follows by rearranging terms. □

Since there are only two possible outcomes for Y, it is easier to write the optimal predictor as follows, applying the definition of conditional probability:

$$f^*(x) = \mathbb{1}\left\{\frac{\mathbb{P}(X = x \mid Y = 1)}{\mathbb{P}(X = x \mid Y = 0)} \geq \frac{[l(1,0) - l(0,0)]\mathbb{P}(Y = 0)}{[l(0,1) - l(1,1)]\mathbb{P}(Y = 1)}\right\}$$

Note, the right-hand-side term appearing in this rewriting of the optimal predictor is independent from x, while the left-hand-side term is important enough to get its own name.

Definition 2.2 (Likelihood Ratio and Test). The likelihood ratio is the ratio of the likelihood functions:

$$\mathcal{L}(x) = \frac{\mathbb{P}(X = x \mid Y = 1)}{\mathbb{P}(X = x \mid Y = 0)}.$$

A likelihood ratio test is a predictor of the form $\mathbb{1}\{\mathcal{L}(x) \geq \eta\}$, for some scalar $\eta > 0$.

It is often useful not to focus directly on the likelihood function but to apply some monotone function to simplify the calculations. In fact, $\mathbb{1}\{\mathcal{L}(x) \geq \eta\} = \mathbb{1}\{h(\mathcal{L}(x)) \geq h(\eta)\}$, for any monotonically increasing function.

Exercise 2.3. Compute the optimal predictor for the binary classification problem, where the loss is $l(a, b) = \mathbb{1}\{a \neq b\}$, and the two labels are equally likely, i.e., $\mathbb{P}(Y = 1) = \mathbb{P}(Y = 0)$.

Exercise 2.4. Compute the optimal predictor for the following binary classification problem. The labels are such that $\mathbb{P}(Y = 1) = 10^{-6}$, while X behaves according to $\mathcal{N}(0, 1)$ if $Y = 0$ or according to $\mathcal{N}(s, 1)$ otherwise, for some $s \in \mathbb{R}$. The loss function is

$$\begin{cases} l(0,0) = 0 \\ l(1,1) = -10^6 \end{cases} \qquad \begin{cases} l(1,0) = 100 \\ l(0,1) = 0 \end{cases}$$

Exercise 2.5. Compute the optimal predictor for the following binary classification problem with 0-1 loss. The two labels have the same probability, i.e., $\mathbb{P}(Y = 1) = \mathbb{P}(Y = 0) = 1/2$; X conditioned on $Y = 0$ is a standard 2-dimensional gaussian, while conditioning on $Y = 1$ is a gaussian centered in $(1, 1)$ with covariance matrix $1/2 \cdot I$ (where I is the two dimensional identity matrix).

There are only four possible outcomes for a binary classifier f:

	$Y = 0$	$Y = 1$
$f(X) = 0$	true negative	false negative
$f(X) = 1$	false positive	true positive

If we normalize with respect to the corresponding populations, we get the following rates that characterize a predictor:

- True Positive Rate: $TPR = \mathbb{P}(f(X) = 1 \mid Y = 1)$, also called power, sensitivity, or recall.
- False Negative Rate: $FNR = 1 - TPR = \mathbb{P}(f(X) = 0 \mid Y = 1)$ also known as type II error or probability of missed detection.
- False Positive Rate: $FPR = \mathbb{P}(f(X) = 1 \mid Y = 0)$, also known as size, type I error, or probability of false alarm.
- True Negative Rate: $TNR = 1 - FPR = \mathbb{P}(f(X) = 0 \mid Y = 0)$, also known as specificity.

Starting from these rates, there are a few metrics that are massively implemented in machine learning:

- Precision: $\mathbb{P}(Y = 1 \mid f(X) = 1)$
- F1-score: F_1 is the harmonic mean of precision and recall.

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- False discovery rate: expected ratio of false positives over the total number of positives.

Suppose we want to maximize the true positive rate subject to a constraint on the false positive rate. Namely, we want to find the predictor that solves:

$$\max TPR \quad \text{subject to} \quad FPR \leq \alpha.$$

The following Theorem (that we do not prove) states that the right thing to do is to look at the likelihood ratio test.

Theorem 2.6 (Neyman-Pearson Lemma). Suppose that the conditional densities $f(x \mid y)$ are continuous. Then the optimal predictor that maximizes TPR with an upper bound on FPR is a likelihood ratio test.

Finally, imagine that you do not have a clear idea of what the right loss function for your problem is, but want to get the best out of the problem at hand. In other words, you may want to understand the better trade-off between recall (TPR) and size (FPR). A helpful tool is to plot for any $\alpha \in [0, 1]$ what is the best TPR achievable with the constraint that $FPR \leq \alpha$ in the FPR-TPR plane.

Remark 2.7. By Theorem 2.6, the ROC curve is given by varying the threshold in the likelihood ratio test from negative to positive infinity.

Exercise 2.8. Plot the ROC curve for the classification problem described in Exercise 2.4.

1.3 Naive Bayes Classifier

Everything worked so far by assuming full knowledge of the joint distribution D over $\mathcal{X} \times \mathcal{Y}$. This is an unrealistic assumption: what's the use of learning if we know everything? We want to estimate the conditional probabilities (i.e., the ones appearing in the left-hand side of the likelihood ratio $\mathcal{L}(x)$), for feature x, that is a d-dimensional object (either continuous or discrete). Instead of trying to learn $\mathbb{P}(X = x \mid Y = y)$ for all possible pairs (x, y), the Naive Bayes Classifier makes the optimistic (and naive) assumption that all the coordinates $X_1, X_2, \ldots, X_d$ of X are independent, conditionally on $Y = y$.

Example 3.1 (Discrete Random Variables). Consider the case in which $Y = \{0, 1\}$, and $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d$ is a discrete set with $|\mathcal{X}_i| = n_i$. A priori, we would need to learn $2 \cdot \prod_i n_i$ numbers, namely all the $\mathbb{P}(X = (x_1, \ldots, x_d) \mid Y = y)$. The naive Bayes classifier assumes that all the coordinates of x are independent given $Y = y$, therefore:

$$\mathbb{P}(X = (x_1, \ldots, x_d) \mid Y = y) = \prod_{i=1}^{d} \mathbb{P}(X_i = x_i \mid Y = y).$$

We must only learn $2 \cdot \sum_i n_i$ numbers. This is an exponentially smaller number!

Example 3.2 (Continuous Random Variables). Consider a multi-class classification setting where $\mathcal{X} = \mathbb{R}^d$ and the loss function is the 0-1. The problem is that we cannot learn in full generality a continuous number of points in a naive way! There is a classical statistical way around it: instead of estimating the probability density function directly, we assume that the underlying distribution has a specific shape, characterized by a few parameters, and we then estimate such parameters! For instance, we can assume that X given $Y = y$ behaves as a multivariate Gaussian distribution (this is the Gaussian Naive Bayes classifier used on the iris dataset). This means that it is characterized by the covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$ and mean $\mu \in \mathbb{R}^d$, and thus we would need to estimate $O(d^2)$ numbers. The naive Bayes classifier assumes that the dimensions are independent, so we would only need to learn d variances and d means, for a total of $O(d)$ numbers. Remember: you have already computed the optimal predictor for this case in Exercise 1.12.

Important: The Naive Bayes Classifier is not one single classifier, but a method to estimate the optimal predictor. In particular, it makes the naive assumption that all the X_i are independent, conditionally on $Y = y$. This assumption may

bring poor results when the data exhibits strong (and asymmetric) correlations, but it avoids the curse of dimensionality.

2 Week 2: Linear Regression

2.1 Warm-Up: Linear Regression in 1-d

Consider the following task: you are given n pairs of numbers $(x_1, y_1), \ldots, (x_n, y_n)$, and the goal is to find the line $y = mx + q$ that minimizes the (empirical) mean square error:

$$\hat{R}(m, q) = \frac{1}{n} \sum_{i=1}^{n} (mx_i + q - y_i)^2.$$

This is a polynomial in m and q, goes to infinity as m and q grow in absolute value, and is thus minimized in the critical value of the gradient. We set to zero the derivative with respect to q:

$$\frac{\partial \hat{R}}{\partial q} = \frac{2}{n} \sum_{i=1}^{n} (mx_i + q - y_i) = 0 \Rightarrow q = \frac{1}{n} \sum_{i=1}^{n} (y_i - mx_i) = \overline{y} - m\overline{x},$$

where $\overline{x} = \sum_i x_i/n$ and $\overline{y} = \sum_i y_i/n$. We move our attention to m, and compute the corresponding derivative:

$$\frac{\partial \hat{R}}{\partial m} = \frac{2}{n} \sum_{i=1}^{n} x_i(mx_i + q - y_i) = 0.$$

By plugging the value $q = \overline{y} - m\overline{x}$ in the last equality, we get:

$$\sum_{i=1}^{n} x_i(mx_i - m\overline{x} + \overline{y} - y_i) = 0 \Rightarrow m = \frac{\sum_{i=1}^{n} x_i(y_i - \overline{y})}{\sum_{i=1}^{n} x_i(x_i - \overline{x})}.$$

While the above formula is complete, there is a cleaner version of it that is usually reported. It can be obtained by massaging the numerator and the denominator:

$$\frac{1}{n} \sum_{i=1}^{n} x_i(y_i - \overline{y}) = \frac{1}{n} \sum_{i=1}^{n} (x_i y_i - \overline{y}x_i - \overline{x}y_i + \overline{xy}) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \overline{x})(y_i - \overline{y})$$

$$\frac{1}{n} \sum_{i=1}^{n} x_i(x_i - \overline{x}) = \frac{1}{n} \sum_{i=1}^{n} (x_i^2 - 2\overline{x}x_i + \overline{x}^2) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \overline{x})^2$$

All in all, we have the classical formula for the one-dimensional ordinary least-squares estimators:

$$\hat{m} = \frac{\sum_{i=1}^{n} (x_i - \overline{x})(y_i - \overline{y})}{\sum_{i=1}^{n} (x_i - \overline{x})^2}, \quad \hat{q} = \overline{y} - \hat{m}\overline{x}.$$

2.2 The Linear Regression Framework

In the one-dimensional setting, we found the linear function (the line) that minimizes the mean squared error. We can generalize this approach in two directions: (i) considering higher dimensions, (ii) considering functions that are linear in the parameters, and not on the points. We are given n points $(x_1, y_1), \ldots, (x_n, y_n)$, where each $x_i \in \mathcal{X}$ and $y_i \in \mathbb{R}$. On the space $\mathcal{X}$, we are given d feature maps $\phi_j : \mathcal{X} \to \mathbb{R}$ which induce functions of the type $f_\theta : \mathcal{X} \to \mathbb{R}$ for $\theta \in \mathbb{R}^d$ as follows:

$$f_\theta(x) = \sum_{j=1}^{d} \phi_j(x) \cdot \theta_j = \phi(x)^T \theta \tag{1}$$

where $\phi(x)$ is the d-dimensional vector whose j^{th} entry is $\phi_j(x)$. Note, this problem is still called linear regression because function $f_\theta(x)$ is not linear in x, but is linear in θ!

Our optimization problem. Our goal is to find $\theta \in \mathbb{R}^d$ that minimizes the mean squared error on the input points with respect to the feature maps $\phi_1, \ldots, \phi_d$. Stated differently, we want to find the vector θ, that minimizes:

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - f_\theta(x_i))^2 = \frac{1}{n} \sum_{i=1}^{n} (y_i - \phi(x_i)^T \theta)^2 = \frac{1}{n} \|y - \Phi\theta\|_2^2.$$

The last version of the mean squared error is more compact as it uses vector and matrix notation. But don't be scared: y is the n-dimensional vector whose i^{th} component is simply y_i. Similarly, Φ is the $n \times d$ matrix obtained by stacking one onto the others the vectors $\phi(x_1)^T, \ldots, \phi(x_n)^T$; in particularly, the generic $\Phi_{i,j}$ element is $\phi_j(x_i)$. Note, the matrix Φ does depend on the x_i's.

Example 2.1 (1-d Linear Regression). If we get back to the one-dimensional example, we see that it is captured by Equation (1), by setting $\phi_0 = 1$, and $\phi_1 = x$.

Example 2.2 (Polynomial Regression). We can generalize the above example and consider polynomial regression so that $f_\theta(x) = \sum_{j=0}^{l} \theta_j x^j$. In general, we can capture l-degree polynomials with $l + 1$ dimensional θ.

Example 2.3 (Fourier Regression). If the function we are trying to estimate look periodic, it may be a good idea to consider as feature maps functions like $\sin(\frac{2\pi x}{T} i)$ and $\cos(\frac{2\pi x}{T} i)$ for varying i.

2.3 Ordinary Least Squares Estimator

It turns out that there is a simple expression for the regression problem, which is called the Ordinary- Least Squares Estimator (OLS):

Theorem 2.4 (OLS Estimator). Let $(x_1, y_1), \ldots, (x_n, y_n)$ be points in $\mathcal{X} \times \mathbb{R}$, and $\phi_1, \ldots, \phi_d$ be functions from $\mathcal{X} \to \mathbb{R}$. If $\Phi^T\Phi$ is invertible, then the Ordinary-Least Squares Estimators

$$\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^T y$$

is a solution to $\min_{\theta \in \mathbb{R}^d} \hat{R}(\theta)$.

Proof. Our goal is to minimize the function $\hat{R}(\theta) = \frac{1}{n}\|y - \Phi\theta\|_2^2$ which is a degree-two polynomial in the coordinates of θ. Moreover it is coercive, meaning that for $\|\theta\|_2 \to \infty$, the value of $\hat{R}$ diverges to $+\infty$. This implies that we need to find the value(s) for which the gradient of $\hat{R}$ is zero. To compute this gradient, let's rewrite more explicitly $\hat{R}$:

$$\begin{aligned}
\hat{R}(\theta) &= \frac{1}{n}(y - \Phi\theta)^T(y - \Phi\theta) \\
&= \frac{1}{n}(y^T y - (\Phi\theta)^T y - y^T\Phi\theta + (\Phi\theta)^T(\Phi\theta)) \\
&= \frac{1}{n}(\|y\|_2^2 - \theta^T\Phi^T y - y^T\Phi\theta + \theta^T\Phi^T\Phi\theta) \\
&= \frac{1}{n}(\|y\|_2^2 - 2\theta^T\Phi^T y + \theta^T\Phi^T\Phi\theta).
\end{aligned}$$

The last equality holds because $\theta^T\Phi^T y$ is a scalar, so it equals its transpose $y^T\Phi\theta$. Starting from this, we can easily compute the gradient of R with respect to θ:

$$\nabla\hat{R}(\theta) = \frac{1}{n}(2\Phi^T\Phi\theta - 2\Phi^T y) = \frac{2}{n}(\Phi^T\Phi\theta - \Phi^T y).$$

Finally, the statement of the theorem holds by setting the gradient to zero, which is equivalent to solving the following equation:

$$\Phi^T\Phi\theta = \Phi^T y.$$

Under the assumption that $\Phi^T\Phi$ is invertible, the system admits a unique solution of the form $\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^T y$. If the gradient has a unique zero and the function is coercive (and convex, which it is), this means that $\hat{\theta}$ minimizes $\hat{R}$. □

A necessary condition for $\Phi^T\Phi$ to be invertible is that $d \leq n$. If this is not the case, in fact, then Φ (an $n \times d$ matrix) has rank at most $n < d$, so its kernel would be non-empty.

2.4 A Geometric Interpretation

There is a nice geometric way of interpreting what the OLS estimator does, in terms of orthogonal projections on suitable subspaces. We start by recalling a fact

from linear algebra. Consider any (finite-dimensional) vector space V, then for any linear subspace $S \subseteq V$, we can define its orthogonal $S^{\perp}$ as the linear subspace of all the vectors $v \in V$ orthogonal to S (a vector v is orthogonal to S if $v^T s = 0$ for all $s \in S$). We can write $V = S \oplus S^{\perp}$, meaning that any $v \in V$ can be written in a unique way as the sum of some $v_{\parallel} \in S$ and $v_{\perp} \in S^{\perp}$. The vector $v_{\parallel}$ is called projection of v onto S.

Proposition 2.5 (OLS as Projection). The linear function $P = \Phi(\Phi^T\Phi)^{-1}\Phi^T$ from $\mathbb{R}^n$ to $\mathbb{R}^n$ is the linear projection onto $im(\Phi)$, the linear subspace generated by the columns of Φ.

Proof. We prove this result in three steps: first, we argue that for any vector $v \in im(\Phi)$, it holds that $Pv = v$. Second, we consider any $v \in im(\Phi)^{\perp}$, and argue that v belongs to the kernel of P. Finally, we prove the statement of the Proposition.

We start with a generic $v \in im(\Phi)$. By definition, vector v is spanned by the column vector of Φ, i.e., $v = \Phi\lambda$ for some coefficient vector $\lambda \in \mathbb{R}^d$. We have the following:

$$Pv = \Phi(\Phi^T\Phi)^{-1}\Phi^T v = \Phi(\Phi^T\Phi)^{-1}\Phi^T(\Phi\lambda) = \Phi(\Phi^T\Phi)^{-1}(\Phi^T\Phi)\lambda = \Phi\lambda = v.$$

Moving to the second point in our proof, consider the generic vector v that is orthogonal to $im(\Phi)$. This means that v is orthogonal to the columns vectors of Φ. This is equivalent to $v \in \ker(\Phi^T)$, i.e., $\Phi^T v = 0$. Therefore, we have the following:

$$Pv = \Phi(\Phi^T\Phi)^{-1}\Phi^T v = \Phi(\Phi^T\Phi)^{-1}0 = 0.$$

We have arrived at the last step of the proof: consider the decomposition of a generic $v \in \mathbb{R}^n$ into $v = v_{\perp} + v_{\parallel}$, with respect to the linear subspace $im(\Phi)$:

$$\begin{aligned} Pv &= P(v_{\perp} + v_{\parallel}) \quad (v = v_{\perp} + v_{\parallel} \text{ where } v_{\parallel} \in im(\Phi), \text{ and } v_{\perp} \in im(\Phi)^{\perp}) \\ &= Pv_{\perp} + Pv_{\parallel} \quad \text{(By linearity)} \\ &= 0 + v_{\parallel} = v_{\parallel} \end{aligned}$$

where the last equality follows from the first two points of the proof. □

This Proposition provides an algorithmic and geometric insight into the OLS estimator formula:

- First we compute the projection of the y vector onto the d-dimensional linear subspace $im(\Phi)$. This projection is $\hat{y} = Py = \Phi(\Phi^T\Phi)^{-1}\Phi^T y$.
- Then, we find $\hat{\theta}$ which are the coordinates of the projected vector $\hat{y}$ with respect to the basis of the linear subspace provided by the column vectors of Φ.

In formula, this corresponds to solving the linear system $\Phi\theta = \hat{y} = \Phi(\Phi^T\Phi)^{-1}\Phi^T y$. If Φ has full column rank, we can left-multiply by $(\Phi^T\Phi)^{-1}\Phi^T$ to get $\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^T y$.

2.5 The Linear Assumption and Linear Regression

Last week, we introduced the framework of decision theory to assess and quantify the quality of a predictor. We now want to look at linear regression from this perspective.

Definition 3.1. We say that a distribution D over $\mathcal{X} \times \mathbb{R}$ respects the linear assumption if there exists a vector $\theta^* \in \mathbb{R}^d$, and d-feature maps $\phi_1, \ldots, \phi_d$ such that:

- $Y = \phi(X)^T\theta^* + \epsilon$
- ϵ is a random variable independent from X such that $\mathbb{E}[\epsilon] = 0$, and $\mathbb{E}[\epsilon^2] = \sigma^2$.

In words, the linear assumption imposes no requirement on $X \in \mathcal{X}$, but strictly constraints the value of Y given X: Y is deterministically specified by $\phi(X)^T\theta^*$, plus an independent and unbiased noise ϵ. In particular, if the variance term σ^2 goes to zero, then the random points (X, Y) will lie closer and closer to the curve $y = \phi(x)^T\theta^*$.

From a geometrical perspective, the curve characterized by $y = \phi(x)^T\theta^*$ is not a linear subspace in the x variable, but only in the $\phi(x)$ variable.

Proposition 3.2 (Optimal Predictor). Under the linear assumption and the squared loss, the optimal predictor is $f^*(x) = \mathbb{E}[Y \mid X = x] = \phi(x)^T\theta^*$.

Proof. We showed last week that the best predictor under the squared loss is $\mathbb{E}[Y \mid X = x]$ so we just need to compute explicitly that conditional expectation. Remember, we are under the linear assumption, therefore conditioning on the random variable X having value x we know that $Y = \phi(x)^T\theta^* + \epsilon$, where the only randomness is given by the noise term ϵ. For any possible realization x of X, we have the following:

$$\mathbb{E}[Y \mid X = x] = \mathbb{E}[\phi(x)^T\theta^* + \epsilon \mid X = x] = \phi(x)^T\theta^* + \mathbb{E}[\epsilon \mid X = x] = \phi(x)^T\theta^*.$$

Note, in the second to last equality we have used that ϵ is independent from X (so $\mathbb{E}[\epsilon \mid X = x] = \mathbb{E}[\epsilon]$), while the last equality descends from the linear assumption, namely that $\mathbb{E}[\epsilon] = 0$. □

Important: The last Proposition tells us that in order to minimize the expected risk under the linear assumption it is enough to know the vector θ^*. Clearly, such vector is unknown, so we need to estimate it from data.

2.6 Estimating θ^*

We prove that OLS estimator does estimate correctly the parameter it is trying to learn: θ^*.

Proposition 3.3. Consider the estimator $\hat{\theta}$ computed on n i.i.d. samples from a distribution D on $\mathcal{X} \times \mathbb{R}$ that satisfies the linear assumption, then:

- The estimator is unbiased: $\mathbb{E}[\hat{\theta}] = \theta^*$
- The variance of the estimator is $\mathbb{V}[\hat{\theta}] = \sigma^2 \mathbb{E}[(\Phi^T \Phi)^{-1}]$

Proof. Fix any realization of the x values of the n i.i.d. samples, i.e., $x_1, \ldots, x_n$. We denote with Y the random vector containing the y values of the samples, and with y a realization of it. Similarly ϵ is the random vector whose entries correspond to the n i.i.d. realizations of the random noise. We have the following:

$$\begin{aligned}
\mathbb{E}[\hat{\theta} \mid X_1 = x_1, \ldots, X_n = x_n] &= \mathbb{E}[(\Phi^T \Phi)^{-1} \Phi^T Y \mid X_1 = x_1, \ldots, X_n = x_n] \\
&= (\Phi^T \Phi)^{-1} \Phi^T \mathbb{E}[Y \mid X_1 = x_1, \ldots, X_n = x_n] \quad (\Phi \text{ only depends on } x_i) \\
&= (\Phi^T \Phi)^{-1} \Phi^T \mathbb{E}[\Phi\theta^* + \epsilon \mid X_1 = x_1, \ldots, X_n = x_n] \\
&= (\Phi^T \Phi)^{-1} (\Phi^T \Phi)\theta^* + (\Phi^T \Phi)^{-1} \Phi^T \mathbb{E}[\epsilon \mid X_1 = x_1, \ldots, X_n = x_n] \\
&= \theta^*,
\end{aligned}$$

where in the last equality we have canceled the noise terms as $\mathbb{E}[\epsilon \mid X_i = x_i] = \mathbb{E}[\epsilon] = 0$ due to independence and $\mathbb{E}[\epsilon_i] = 0$. Since the conditional expectation is θ^* for any $x_1, \ldots, x_n$, the total expectation $\mathbb{E}[\hat{\theta}] = \mathbb{E}_X[\mathbb{E}[\hat{\theta} \mid X]]$ is also θ^*.

We move our attention to the variance and study the difference between $\hat{\theta}$ and θ^*. Conditioned on $X_1, \ldots, X_n$:

$$\hat{\theta} - \theta^* = (\Phi^T \Phi)^{-1} \Phi^T Y - \theta^* = (\Phi^T \Phi)^{-1} \Phi^T (\Phi\theta^* + \epsilon) - \theta^* = (\Phi^T \Phi)^{-1} \Phi^T \epsilon.$$

The variance of a generic random vector Z is a matrix defined as $\mathbb{V}[Z] = \mathbb{E}[(Z - \mathbb{E}[Z])(Z - \mathbb{E}[Z])^T]$. We first compute the conditional variance $\mathbb{V}[\hat{\theta} \mid X]$:

$$\begin{aligned}
\mathbb{V}[\hat{\theta} \mid X] &= \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta} \mid X])(\hat{\theta} - \mathbb{E}[\hat{\theta} \mid X])^T \mid X] \\
&= \mathbb{E}[(\hat{\theta} - \theta^*)(\hat{\theta} - \theta^*)^T \mid X] \\
&= \mathbb{E}[((\Phi^T \Phi)^{-1} \Phi^T \epsilon)((\Phi^T \Phi)^{-1} \Phi^T \epsilon)^T \mid X] \\
&= \mathbb{E}[(\Phi^T \Phi)^{-1} \Phi^T \epsilon\epsilon^T \Phi (\Phi^T \Phi)^{-1} \mid X] \quad ((\Phi^T \Phi)^{-1} \text{ is symmetric}) \\
&= (\Phi^T \Phi)^{-1} \Phi^T \mathbb{E}[\epsilon\epsilon^T \mid X] \Phi (\Phi^T \Phi)^{-1} \\
&= (\Phi^T \Phi)^{-1} \Phi^T (\sigma^2 I) \Phi (\Phi^T \Phi)^{-1} \quad (\epsilon \text{ is i.i.d., independent of } X) \\
&= \sigma^2 (\Phi^T \Phi)^{-1} \Phi^T \Phi (\Phi^T \Phi)^{-1} = \sigma^2 (\Phi^T \Phi)^{-1}.
\end{aligned}$$

Note, we crucially rely on $\mathbb{E}[\epsilon\epsilon^T] = \sigma^2 I$ because the ϵ_i are independent (so $\mathbb{E}[\epsilon_i\epsilon_j] = 0$ for $i \neq j$) and $\mathbb{E}[\epsilon_i^2] = \sigma^2$. The total variance is the expectation of the conditional variance (since $\mathbb{E}[\hat{\theta} \mid X]$ is constant θ^*):

$$\mathbb{V}[\hat{\theta}] = \mathbb{E}_X[\mathbb{V}[\hat{\theta} \mid X]] = \mathbb{E}_X[\sigma^2(\Phi^T\Phi)^{-1}] = \sigma^2\mathbb{E}[(\Phi^T\Phi)^{-1}].$$

□

Remark 3.4 (Variance Rate). Let $\hat{\Sigma} = \frac{1}{n}\Phi^T\Phi$. Then $\mathbb{V}[\hat{\theta}] = \frac{\sigma^2}{n}\mathbb{E}[\hat{\Sigma}^{-1}]$. Typically, by the law of large numbers, $\hat{\Sigma}$ converges to $\Sigma = \mathbb{E}[\phi(X)\phi(X)^T]$, a deterministic matrix. This means that the variance of the estimator decreases linearly in $1/n$.

Important: We are trying to estimate the predictor $\phi(X)^T\theta^*$ which minimizes the expected risk $\mathbb{E}[(Y - \phi(X)^T\theta^*)^2]$, with the predictor $\phi(X)^T\hat{\theta}$, which minimizes the empirical mean-squared error: $\frac{1}{n}\sum_{i=1}^n (y_i - \phi(x_i)^T\theta)^2$. Since we do not have access to the distribution D, we are using the n samples as a proxy for it!

2.7 The Risk of the OLS Estimator

We can compute explicitly the expected risk $R(\theta)$ associated with any predictor of the form $f_\theta(x) = \phi(x)^T\theta$.

Proposition 3.5. Under the linear assumption, for any (non-random) $\theta \in \mathbb{R}^d$, we have

$$R(\theta) = (\theta^* - \theta)^T\Sigma(\theta^* - \theta) + R(\theta^*)$$

where the matrix Σ is defined as $\mathbb{E}[\phi(X)\phi(X)^T]$. Moreover, $R(\theta^*) = \sigma^2$.

Proof. Consider a generic vector θ, we have the following chain of inequalities:

$$\begin{aligned}
R(\theta) &= \mathbb{E}[(Y - \phi(X)^T\theta)^2] \quad \text{(By definition of expected risk)} \\
&= \mathbb{E}[(\phi(X)^T\theta^* + \epsilon - \phi(X)^T\theta)^2] \quad \text{(By the linear assumption)} \\
&= \mathbb{E}[(\phi(X)^T(\theta^* - \theta) + \epsilon)^2] \\
&= \mathbb{E}[(\phi(X)^T(\theta^* - \theta))^2] + 2\mathbb{E}[\epsilon(\phi(X)^T(\theta^* - \theta))] + \mathbb{E}[\epsilon^2] \\
&= \mathbb{E}[(\phi(X)^T(\theta^* - \theta))^2] + 2\mathbb{E}[\epsilon]\mathbb{E}[\phi(X)^T(\theta^* - \theta)] + \mathbb{E}[\epsilon^2] \quad \text{(X and } \epsilon \text{ are independent)} \\
&= \mathbb{E}[(\phi(X)^T(\theta^* - \theta))^2] + \sigma^2 \quad \text{(By assumption, } \mathbb{E}[\epsilon] = 0, \mathbb{E}[\epsilon^2] = \sigma^2) \\
&= \mathbb{E}[((\theta^* - \theta)^T\phi(X))(\phi(X)^T(\theta^* - \theta))] + \sigma^2 \\
&= (\theta^* - \theta)^T\mathbb{E}[\phi(X)\phi(X)^T](\theta^* - \theta) + \sigma^2 \quad (\theta, \theta^* \text{ are deterministic}) \\
&= (\theta^* - \theta)^T\Sigma(\theta^* - \theta) + \sigma^2
\end{aligned}$$

Note, if $\theta = \theta^*$, then the first term is zero, and we are left with $R(\theta^*) = \sigma^2$. This concludes the proof. □

Proposition 3.6. Under the linear model, the expected risk of the OLS estimator $\hat{\theta}$ (which is a random variable depending on the n samples) is

$$\mathbb{E}[R(\hat{\theta})] = R(\theta^*) + \frac{\sigma^2}{n}\mathbb{E}[\mathrm{Tr}(\Sigma\hat{\Sigma}^{-1})]$$

where the outer expectation $\mathbb{E}[\cdot]$ is over the n samples $(X_1, Y_1), \ldots, (X_n, Y_n)$ used to compute $\hat{\theta}$, and $\hat{\Sigma} = \frac{1}{n}\Phi^T\Phi$.

Proof. The OLS estimator $\hat{\theta}$ is a random variable that depends on n i.i.d. samples from the linear model, we denote such samples with $(\mathbf{X}, \mathbf{Y})$. When we compute the expected risk $\mathbb{E}[R(\hat{\theta})]$, we are taking the expectation with respect to a new sample (X, Y) (where $Y = \phi(X)^T\theta^* + \epsilon$) AND with respect to the n samples $(\mathbf{X}, \mathbf{Y})$ that define $\hat{\theta}$.

We use Proposition 3.5, with θ replaced by the random variable $\hat{\theta}$.

$$R(\hat{\theta}) = (\theta^* - \hat{\theta})^T\Sigma(\theta^* - \hat{\theta}) + R(\theta^*)$$

This $R(\hat{\theta})$ is still a random variable (depending on $(\mathbf{X}, \mathbf{Y})$). We now take the expectation w.r.t. $(\mathbf{X}, \mathbf{Y})$:

$$\mathbb{E}[R(\hat{\theta})] = \mathbb{E}[(\theta^* - \hat{\theta})^T\Sigma(\theta^* - \hat{\theta})] + R(\theta^*)$$

Let's analyze the term $\mathbb{E}[(\theta^* - \hat{\theta})^T\Sigma(\theta^* - \hat{\theta})]$. We use the property $\mathrm{Tr}(A) = A$ for a scalar A, and $\mathrm{Tr}(ABC) = \mathrm{Tr}(CAB)$.

$$\begin{aligned}
\mathbb{E}[(\theta^* - \hat{\theta})^T\Sigma(\theta^* - \hat{\theta})] &= \mathbb{E}[\mathrm{Tr}((\theta^* - \hat{\theta})^T\Sigma(\theta^* - \hat{\theta}))] \\
&= \mathbb{E}[\mathrm{Tr}(\Sigma(\theta^* - \hat{\theta})(\theta^* - \hat{\theta})^T)] \quad (\mathrm{Tr}(AB) = \mathrm{Tr}(BA)) \\
&= \mathrm{Tr}(\Sigma \cdot \mathbb{E}[(\theta^* - \hat{\theta})(\theta^* - \hat{\theta})^T]) \quad (\text{Linearity of Trace and Expectation}) \\
&= \mathrm{Tr}(\Sigma \cdot \mathbb{V}[\hat{\theta}]) \quad (\text{Since } \mathbb{E}[\hat{\theta}] = \theta^*) \\
&= \mathrm{Tr}(\Sigma \cdot \sigma^2\mathbb{E}[(\Phi^T\Phi)^{-1}]) \quad (\text{From Prop 3.3}) \\
&= \sigma^2\mathbb{E}[\mathrm{Tr}(\Sigma(\Phi^T\Phi)^{-1})] \quad (\text{Linearity}) \\
&= \sigma^2\mathbb{E}[\mathrm{Tr}(\Sigma(n\hat{\Sigma})^{-1})] = \frac{\sigma^2}{n}\mathbb{E}[\mathrm{Tr}(\Sigma\hat{\Sigma}^{-1})]
\end{aligned}$$

This concludes the proof. □

Before concluding, let's highlight the differences between Σ and $\hat{\Sigma}$. Σ is a deterministic matrix, as it is given by the expected value of $\phi(X)\phi(X)^T$, with respect to a single sample (X,Y). The matrix $\hat{\Sigma}$ is instead a random variable, as it depends on the n i.i.d. samples represented in the vectors $(\mathbf{X}, \mathbf{Y})$, in particular, $\hat{\Sigma} = \frac{1}{n}\Phi(\mathbf{X})^T\Phi(\mathbf{X})$.

3 Week 3: Linear Regression (continued)

3.1 Introduction

Last week, we studied the linear regression problem and found a satisfying closed formula. We recall the setting. We are given n points $(x_1, y_1), \ldots, (x_n, y_n)$ where each $x_i \in \mathcal{X}$, and $y_i \in \mathbb{R}$. On the space $\mathcal{X}$, we are given d feature maps $\phi_j : \mathcal{X} \to \mathbb{R}$, which induce functions of the type $f_\theta : \mathcal{X} \to \mathbb{R}$ for $\theta \in \mathbb{R}^d$ as follows:

$$f_\theta(x) = \sum_{j=1}^{d} \phi_j(x) \cdot \theta_j = \phi(x)^T \theta,$$

where $\phi(x)$ is the d-dimensional vector whose j^{th} entry is $\phi_j(x)$. Note, this problem is called linear regression because the function $f_\theta(x)$ is linear in θ (so, possibly, not in x)!

Our optimization problem. Our goal is to find $\theta \in \mathbb{R}^d$ that minimizes the mean squared error on the input points with respect to the feature maps $\phi_1, \ldots, \phi_d$. Stated differently, we want to find the vector θ, that minimizes:

$$\hat{R}(\theta) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - f_\theta(x_i))^2 = \frac{1}{2n} \sum_{i=1}^{n} (y_i - \phi(x_i)^T \theta)^2 = \frac{1}{2n} \|y - \Phi\theta\|_2^2.$$

We have observed that the solution of the minimization problem is given by the vector $\hat{\theta} \in \mathbb{R}^d$ under the assumption that $\Phi^T \Phi$ is invertible.

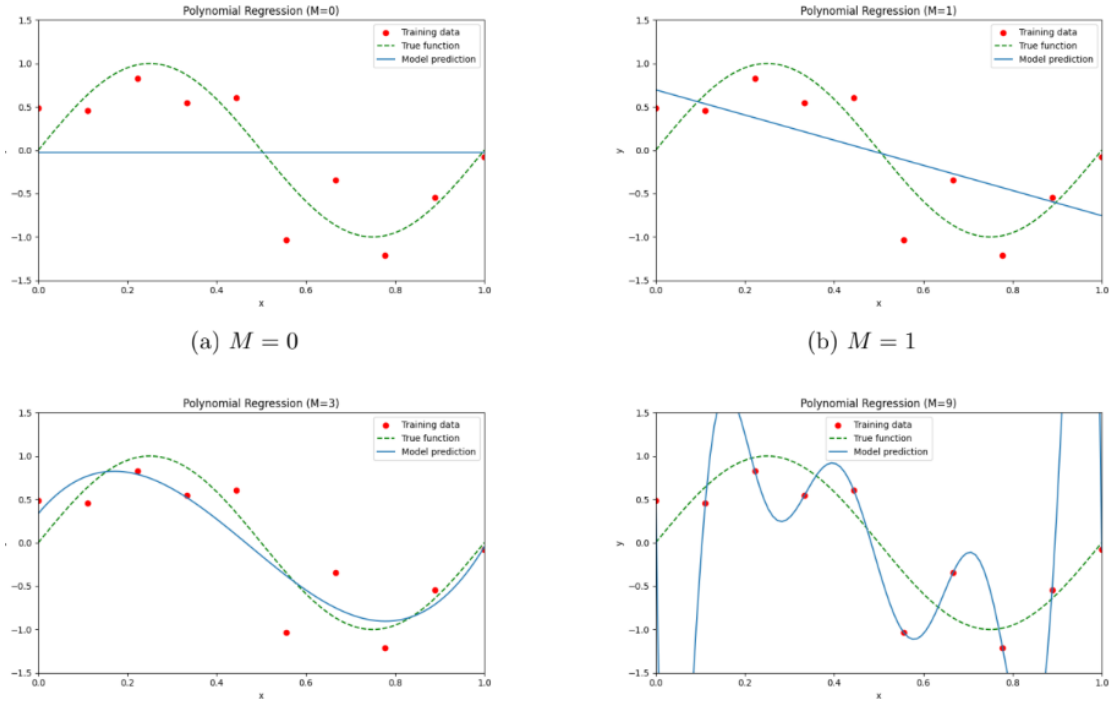
$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y.$$

3.2 Polynomial Regression and Overfitting

A powerful choice of the feature maps is given by polynomials. In this case, we have $\phi_0(x) = 1$, $\phi_1(x) = x, \ldots, \phi_M(x) = x^M$, so that the generic function $f_\theta(x)$ looks like $f_\theta(x) = \sum_{i=0}^{M} \theta_i x^i$. In particular, with $\theta \in \mathbb{R}^{M+1}$ we capture all polynomials of degree up to M. Since we can actually choose M, it is natural to try different ones. Note, if we have n points and set $M = n - 1$, then we will suffer zero error, as it is generally possible to find a polynomial of degree $n - 1$ (which has n coefficients) that passes through n points.

By increasing the degree M, we observe the following phenomenon, which is fairly common in machine learning models: as the complexity of the underlying model increases (in this case, the degree M of the polynomial family considered as predictors), the training error (i.e., the error made on the portion of the data that is actually used to compute $\hat{\theta}$) decreases.

However, we should not forget that our final goal is not to minimize the training error, but to make meaningful predictions on future data points. This is called generalization and is the core of machine learning. If the complexity of the model increases "too much", then the prediction may get too tailored to the data in the training set, and the predictor may lose generalization power. This is called overfitting.



(a) $M = 0$ (b) $M = 1$

(c) $M = 3$ (d) $M = 9$

Figure 1: Predicted curves for various degrees M

From that Figure, it seems like the sweet spot for the complexity of the model, given the (tiny) amount of data we have, is $M = 6$ or 7. There are some standard tools to tackle overfitting. We present here some of them that are well suited for linear regression.

Adding more data points. A good rule of thumb is that more data points buy you the possibility of using more complex models. If we stick to the polynomial regression example, having $n = 100$ points (as opposed to the $n = 10$), allows us to get a good generalization error while exploiting the additional flexibility of using the polynomials of degree 9.

Adding a Regularizer. Another technique consists in adding a regularization

term to the objective that is minimized in the training. Namely, in linear regression, instead of minimizing $\hat{R}(\theta) = \frac{1}{2n}\|y - \Phi\theta\|_2^2$, we aim at minimizing $\hat{R}_{\text{reg}}(\theta) = \frac{1}{2n}\|y - \Phi\theta\|_2^2 + \lambda\Psi(\theta)$, for some regularization function Ψ, and parameter λ.

There are three natural choices of regularizer:

- **Ridge regularizer:** $\Psi(\theta) = \|\theta\|_2^2$. Useful to force "small" values of θ. It has the desirable feature of being convex and differentiable.
- **Lasso regularizer:** $\Psi(\theta) = \|\theta\|_1$. It may force θ to set to zero some coordinates, which is good as it implicitly performs "feature selections".
- **Elastic Net Regularizer:** which is a convex combination of Lasso and Ridge.

Once we decide which regularizer to use, it is still important to set the right hyperparameter λ.

3.3 The Computational Complexity of the OLS Estimator

We now want to understand the computational complexity of computing the closed formula of $\hat{\theta}$, assuming that computing the generic feature map ϕ_j takes constant time. We need to perform the following operations:

- Compute the d feature maps ϕ_j on each one of the n points: $O(nd)$ time steps
- Compute each one of the $d \times d$ entries of $\Phi^T\Phi$, each one given by the scalar product of two n-dimensional vectors, for an overall running time of $O(nd^2)$
- Compute the inverse of $\Phi^T\Phi$, which, in general, takes $O(d^3)$ time steps (e.g., with Gaussian Elimination)
- Compute each one of the d entries of $\Phi^T y$, each one given by the scalar product of two n-dimensional vectors, for an overall running time of $O(nd)$
- Compute the product of the $d \times d$ matrix $(\Phi^T\Phi)^{-1}$, and the d dimensional vector $\Phi^T y$, this takes $O(d^2)$ time steps

Proposition 3.1. The computational cost of computing with the closed formula is $O(nd^2 + d^3)$.

Remark 3.2. On top of the computational complexity, there is a numerical aspect of the problem that we are ignoring. Namely, inverting ill-conditioned matrices may yield undesirable results. The natural question, as computer scientists, is then whether we can get a "good enough" solution with less computational burden.

3.4 Gradient Descent

A standard approach in optimization (and in machine learning) is to follow gradient descent! Consider the generic optimization task of minimizing a differentiable function F over $\mathbb{R}^d$, the gradient descent algorithm works as follows:

1. Start with an arbitrary $\theta_0 \in \mathbb{R}^d$
2. For any $t = 1, 2, \ldots$
   (a) Compute the gradient $\nabla F(\theta_{t-1})$
   (b) Define $\theta_t = \theta_{t-1} - \gamma_t \nabla F(\theta_{t-1})$

The sequence of γ_t is called the learning rate (or stepsize) of the algorithm. Gradient Descent is typically meaningful when there is no distinction between local and global minima. In general, there is indeed always the risk that gradient descent will get trapped in some local minimum, that may be arbitrarily far from the best one. A simple property that avoids this is convexity.

Definition 4.1. A function $F : \mathbb{R}^d \to \mathbb{R}$ is convex if, for any point x, y and $\alpha \in [0, 1]$, the following inequality holds true:

$$F(\alpha x + (1 - \alpha)y) \leq \alpha F(x) + (1 - \alpha)F(y).$$

A function F is concave if -F is convex.

An easy corollary of the definition is that for convex functions (defined on the whole $\mathbb{R}^d$) any local minima is also a global one! Furthermore, there is a simple analytical way of assessing whether a function is convex, by just looking at its Hessian H. A function in $C^2(\mathbb{R}^d)$ is convex if and only if its Hessian is positive (semi-)definite. Similar results hold for concavity/maximization and negative definiteness.

Definition 4.2. A symmetric matrix A is positive semi-definite if $v^T Av \geq 0$ for any vector v. If the inequality is strict for $v \neq 0$, then A is positive definite. A matrix A is called negative (semi-)definite, if -A if positive (semi-)definite.

Proposition 4.3. Let A be a symmetric matrix, and let λ_{max} be the eigenvalue with the largest absolute value. Then for any vector v, it holds $|v^T Av| \leq |\lambda_{max}| \cdot \|v\|_2^2$.

Proof. The matrix is symmetric, so it is possible to diagonalize it. Let $v_1, \ldots, v_d$ be a basis of orthonormal eigenvectors, corresponding to eigenvalues $\lambda_1, \ldots, \lambda_d$. We

can rewrite v according to this new basis: $v = \sum_i \alpha_i v_i$. We have the following:

$$\begin{aligned} v^T A v &= (\sum_i \alpha_i v_i^T) A (\sum_j \alpha_j v_j) \\ &= (\sum_i \alpha_i v_i^T)(\sum_j \alpha_j \lambda_j v_j) \quad \text{(Definition of eigenvectors)} \\ &= \sum_{i,j} \alpha_i \alpha_j \lambda_j v_i^T v_j \\ &= \sum_i \alpha_i^2 \lambda_i \quad (v_i \text{ form orthonormal basis}) \\ &\leq \max_i |\lambda_i| \cdot \sum_i \alpha_i^2 = |\lambda_{\max}| \cdot \|v\|_2^2. \end{aligned}$$

Note, in the last inequality, we are using that $\|v\|_2^2 = \sum_i \alpha_i^2$. □

3.5 Gradient Descent for Linear Regression

Let's see what this algorithm would look like for linear regression, where the objective function we are trying to minimize is $\hat{R}(\theta) = \frac{1}{2n}\|y - \Phi\theta\|_2^2$. We already know that $\nabla\hat{R}(\theta) = \frac{1}{n}(\Phi^T\Phi\theta - \Phi^T y)$ while the hessian is $H = \frac{1}{n}\Phi^T\Phi$. It is a simple exercise to show that $\hat{R}$ is convex, and thus gradient descent would converge to the global minimum (if the step-size is appropriate).

Proposition 4.4. The Hessian matrix $H = \frac{1}{n}\Phi^T\Phi$ is symmetric, and its eigenvalues are non-negative. If $\Phi^T\Phi$ is invertible, its eigenvalues are positive $0 < \mu = \lambda_1 \leq \cdots \leq \lambda_d = L$. Therefore $\hat{R}$ is convex.

Proof. We study the matrix $\Phi^T\Phi$. It is symmetric as $(\Phi^T\Phi)^T = \Phi^T(\Phi^T)^T = \Phi^T\Phi$. Symmetry implies diagonalizability. Moreover for any eigenvector v and corresponding eigenvalue λ it holds:

$$(\Phi^T\Phi)v = \lambda v \Rightarrow \|\Phi v\|_2^2 = v^T\Phi^T\Phi v = v^T(\lambda v) = \lambda\|v\|_2^2 \Rightarrow \lambda = \frac{\|\Phi v\|_2^2}{\|v\|_2^2} \geq 0.$$

In the above inequality, we prove that $v^T H v \geq 0$ for any v, meaning that H is positive semi-definite; this implies convexity. If $\Phi^T\Phi$ is invertible, $\Phi v \neq 0$ for $v \neq 0$, so $\lambda > 0$. □

The gradient descent algorithm for linear regression thus works as follows:

1. Set the initial θ_0 in an arbitrary way
2. For t = 1, . . . , N,

3. Compute $\theta_t = \theta_{t-1} - \gamma_t \nabla \hat{R}(\theta_{t-1}) = \theta_{t-1} - \gamma_t \frac{1}{n}(\Phi^T \Phi \theta_{t-1} - \Phi^T y)$

We massage the formula for the generic term, setting $\gamma_t = \gamma$ for simplicity and $\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$:

$$\begin{aligned}
\theta_t - \hat{\theta} &= \theta_{t-1} - \gamma \frac{1}{n}(\Phi^T \Phi \theta_{t-1} - \Phi^T y) - \hat{\theta} \\
&= \theta_{t-1} - \gamma H \theta_{t-1} + \gamma \frac{1}{n} \Phi^T y - \hat{\theta} \\
&= (I - \gamma H)\theta_{t-1} + \gamma H \hat{\theta} - \hat{\theta} \quad (\text{Since } \frac{1}{n}\Phi^T y = H\hat{\theta}) \\
&= (I - \gamma H)\theta_{t-1} - (I - \gamma H)\hat{\theta} \\
&= (I - \gamma H)(\theta_{t-1} - \hat{\theta}) \\
&= (I - \gamma H)^t (\theta_0 - \hat{\theta}).
\end{aligned}$$

If we now look at the distances, we get:

$$\|\theta_t - \hat{\theta}\|_2^2 = (\theta_0 - \hat{\theta})^T (I - \gamma H)^{2t} (\theta_0 - \hat{\theta}) \quad ((I - \gamma H) \text{ is symmetric})$$

$$\begin{aligned}
\hat{R}(\theta_t) - \hat{R}(\hat{\theta}) &= \frac{1}{2n}(\theta_t - \hat{\theta})^T \Phi^T \Phi (\theta_t - \hat{\theta}) = \frac{1}{2}(\theta_t - \hat{\theta})^T H (\theta_t - \hat{\theta}) \\
&= \frac{1}{2}((I - \gamma H)^t (\theta_0 - \hat{\theta}))^T H ((I - \gamma H)^t (\theta_0 - \hat{\theta})) \\
&= \frac{1}{2}(\theta_0 - \hat{\theta})^T (I - \gamma H)^t H (I - \gamma H)^t (\theta_0 - \hat{\theta}) \\
&= \frac{1}{2}(\theta_0 - \hat{\theta})^T (I - \gamma H)^{2t} H (\theta_0 - \hat{\theta}),
\end{aligned}$$

where the last equality follows by observing that H commutes with $(I - \gamma H)$. We now have all the ingredients to formally study the convergence rate of gradient descent. To this end, recall that μ and L are the smallest, respectively, largest eigenvalues of H. Denote with $\kappa = L/\mu \geq 1$ the condition number.

Theorem 4.5. Consider gradient descent for linear regression with learning rate $\gamma_t = \frac{1}{L}$. The following bounds hold:

- $\|\theta_t - \hat{\theta}\|_2^2 \leq (1 - \frac{1}{\kappa})^{2t} \|\theta_0 - \hat{\theta}\|_2^2$ (Convergence in θ)
- $\hat{R}(\theta_t) - \hat{R}(\hat{\theta}) \leq (1 - \frac{1}{\kappa})^{2t} (\hat{R}(\theta_0) - \hat{R}(\hat{\theta}))$ (Convergence in value)
- $\hat{R}(\theta_t) - \hat{R}(\hat{\theta}) \leq \frac{L}{4t} \|\theta_0 - \hat{\theta}\|_2^2$ (Convergence in value w.r.t. initial θ error)

Proof. We know that $\|\theta_t - \hat{\theta}\|_2^2 = (\theta_0 - \hat{\theta})^T (I - \gamma H)^{2t} (\theta_0 - \hat{\theta})$, so Proposition 4.3 tells us that we need to look at the eigenvalues of $(I - \gamma H)^{2t}$. The eigenvalues of $I - \gamma H$ are of the form $(1 - \gamma\lambda_i)$, for λ_i eigenvalue of H. Setting $\gamma = \frac{1}{L}$ yields that the eigenvalues of $I - \gamma H$ are of the form $1 - \lambda_i/L$. Since $0 \leq \lambda_i \leq L$, these eigenvalues are in $[0, 1 - \mu/L] = [0, 1 - 1/\kappa]$. The largest eigenvalue of $(I - \gamma H)^{2t}$ is thus $(1 - 1/\kappa)^{2t}$. Applying Proposition 4.3:

$$\|\theta_t - \hat{\theta}\|_2^2 \leq \max_i (1 - \frac{\lambda_i}{L})^{2t} \|\theta_0 - \hat{\theta}\|_2^2 = (1 - \frac{1}{\kappa})^{2t} \|\theta_0 - \hat{\theta}\|_2^2.$$

(The rest of the proof is omitted for brevity but follows from similar spectral analysis.) □

Consider now the running time of gradient descent:

- Computing $\Phi^T \Phi$ costs $O(nd^2)$ time steps
- Computing $\Phi^T y$ costs $O(nd)$ time steps
- The running time of the generic step is $O(d^2)$
- Overall the running time is $O(nd^2 + d^2 N)$, where N is the number of iterations

Corollary 4.6. After $N = \frac{\kappa}{2} \log(\frac{1}{\epsilon})$ iterations of gradient descent, it holds that $\|\theta_N - \hat{\theta}\|_2^2 \leq \epsilon \|\theta_0 - \hat{\theta}\|_2^2$.

Proof. We have $(1 - \frac{1}{\kappa})^{2t} \leq (e^{-1/\kappa})^{2t} = e^{-2t/\kappa}$. We thus want to find t such that $e^{-2t/\kappa} \leq \epsilon$, we apply the log to both terms $(-2t/\kappa \leq \log \epsilon)$ and get $t \geq -\frac{\kappa}{2} \log \epsilon = \frac{\kappa}{2} \log \frac{1}{\epsilon}$. □

All in all, we have proved that the computational complexity of decreasing the error by a factor ϵ is $O(nd^2 + d^2 \kappa \log \frac{1}{\epsilon})$. Instead of computing $\Phi^T \Phi$ in advance (it takes $O(nd^2)$), one can compute the gradient from scratch at each time step, for a running time of $O(nd)$. With this approach, the bound becomes $O(ndN) = O(\kappa \log \frac{1}{\epsilon} \cdot nd)$.

3.6 Stochastic Gradient Descent

The downside of gradient descent is that a typical iteration of the algorithm entails a pass through the whole dataset. This phenomenon does not only happen for linear regression, but for all loss function of the forms $L(\theta) = \sum_i l_i(\theta)$, where l_i is the loss associated to the i^{th} sample. Indeed, the gradient of L is given by the sum of the n gradients of the l_i terms, each depending on one single data point.

A natural way of getting around this computational bottleneck is to use a random proxy for ∇L. For instance, we could use the random variable g defined below:

$$g(\theta_t) = \nabla l_{i_t}(\theta_t), \quad \text{where } i_t \text{ is a point chosen uniformly at random}$$

It is a simple exercise to see that $\mathbb{E}[g(\theta_t)] = \nabla L(\theta)$, meaning that in expectation, the random vector g points in the right direction. To improve the variance of the random variable g, we could use the so-called minibatch approach, namely, instead of drawing one point i_t uniformly at random, we could sample some of them and use them to get a more robust estimate of the gradient in θ_t.

4 Week 4: Logistic and Softmax Regression

4.1 Classification with "Linear" Regression: Logistic Regression

We want to use our insight on linear regression for binary classification, i.e., in the supervised learning setting where the label set Y consists of only two classes, let's say $\{0, 1\}$. We can use linear functions as a "score" for the likelihood of a certain x to be associated with the label 1. To this end, we introduce the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid function maps $\mathbb{R}$ in $[0, 1]$, meaning that we can interpret its output as probabilities. Moreover, its derivative has a very convenient form:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z))$$

In this section, we predict the probabilities with the following rule:

$$h_\theta(x) = \sigma(\phi(x)^T \theta) = \frac{1}{1 + e^{-\phi(x)^T \theta}}$$

This is called a logistic function, and ideally tells us how likely a given class is to realize! Given the prediction made by the logistic predictor, it is immediate to design the corresponding prediction: if $h_\theta(x) \geq 1/2$ then the predicted label is 1, otherwise, it is 0. Now the question is about what is a good loss for probabilities. Given two probabilities p and q over a discrete support, we define the Kullback-Leibler divergence (KL in short) as

$$D_{KL}(p \| q) = \sum_i p_i \log \frac{p_i}{q_i}$$

where p_i and q_i are the probabilities of observing the i^{th} outcome, and we adopt the convention that $0 \times \log \frac{0}{q} = 0$ and $p \times \log \frac{p}{0} = \infty$. We have the following properties:

- The KL divergence is non-negative, and it is equal to zero only if $p = q$.
- It does not depend on the actual values of the support of the distribution.
- It weights the log of the probabilities by the probability of each class.

This gives us a tentative objective: minimize the KL divergence between the true conditional probability $p(y \mid x)$ and our predicted $h_\theta(x)$. Let's now look at the

empirical error over n samples of the form (x_i, y_i), where $y_i \in \{0, 1\}$. For point i, the true distribution is $p_i = (1 - y_i, y_i)$ and the predicted one is $q_i = (1 - h_\theta(x_i), h_\theta(x_i))$. We are measuring the KL divergence:

$$\begin{aligned} D_{KL}(p_i \| q_i) &= \sum_{k \in \{0,1\}} p_i(k) \log \frac{p_i(k)}{q_i(k)} \\ &= y_i \log \frac{y_i}{h_\theta(x_i)} + (1 - y_i) \log \frac{1 - y_i}{1 - h_\theta(x_i)} \end{aligned}$$

The total empirical risk is $\frac{1}{n} \sum_{i=1}^{n} D_{KL}(p_i \| q_i)$:

$$\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log \frac{y_i}{h_\theta(x_i)} + (1 - y_i) \log \frac{1 - y_i}{1 - h_\theta(x_i)} \right]$$

$$= \frac{1}{n} \sum_{i=1}^{n} \underbrace{[y_i \log y_i + (1 - y_i) \log(1 - y_i)]}_{\text{Does not depend on } \theta} - \frac{1}{n} \sum_{i=1}^{n} [y_i \log h_\theta(x_i) + (1 - y_i) \log(1 - h_\theta(x_i))]$$

The first terms do not depend on the predictor, so our goal is to find θ that minimizes the negative log-likelihood, which is also called the cross-entropy:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log h_\theta(x_i) + (1 - y_i) \log(1 - h_\theta(x_i))]$$

Alternatively, we could aim at maximizing the (rescaled) log-likelihood:

$$\hat{l}(\theta) = \frac{1}{n} \sum_{i=1}^{n} [y_i \log h_\theta(x_i) + (1 - y_i) \log(1 - h_\theta(x_i))] = -J(\theta).$$

Unfortunately, there is no closed formula for finding $\arg\max \hat{l}(\theta)$, but we can always resort to (stochastic) gradient descent (or ascent for maximization). Let's compute

the gradient of $\hat{l}(\theta)$ w.r.t. a single θ_j:

$$\frac{\partial \hat{l}}{\partial \theta_j} = \frac{1}{n}\sum_{i=1}^{n}\left[\frac{y_i}{h_\theta(x_i)}\frac{\partial h_\theta(x_i)}{\partial \theta_j} + \frac{1-y_i}{1-h_\theta(x_i)}\frac{\partial(1-h_\theta(x_i))}{\partial \theta_j}\right]$$

$$\frac{\partial h_\theta(x_i)}{\partial \theta_j} = \frac{\partial \sigma(z_i)}{\partial z_i}\frac{\partial z_i}{\partial \theta_j} \quad (\text{where } z_i = \phi(x_i)^T\theta)$$

$$= \sigma(z_i)(1-\sigma(z_i))\cdot\phi_j(x_i) = h_\theta(x_i)(1-h_\theta(x_i))\phi_j(x_i)$$

$$\frac{\partial \hat{l}}{\partial \theta_j} = \frac{1}{n}\sum_{i=1}^{n}\left[\frac{y_i}{h_\theta(x_i)}h_\theta(x_i)(1-h_\theta(x_i))\phi_j(x_i) - \frac{1-y_i}{1-h_\theta(x_i)}h_\theta(x_i)(1-h_\theta(x_i))\phi_j(x_i)\right]$$

$$= \frac{1}{n}\sum_{i=1}^{n}[y_i(1-h_\theta(x_i)) - (1-y_i)h_\theta(x_i)]\phi_j(x_i)$$

$$= \frac{1}{n}\sum_{i=1}^{n}[y_i - y_i h_\theta(x_i) - h_\theta(x_i) + y_i h_\theta(x_i)]\phi_j(x_i)$$

$$= \frac{1}{n}\sum_{i=1}^{n}[y_i - h_\theta(x_i)]\phi_j(x_i)$$

Let's now look at the second derivative,

$$\frac{\partial^2 \hat{l}}{\partial \theta_j \partial \theta_k} = \frac{1}{n}\sum_{i=1}^{n} -\frac{\partial h_\theta(x_i)}{\partial \theta_k}\phi_j(x_i) = -\frac{1}{n}\sum_{i=1}^{n} h_\theta(x_i)(1-h_\theta(x_i))\phi_k(x_i)\phi_j(x_i)$$

In matrix form, this means that the Hessian is $H = -\frac{1}{n}\sum_{i=1}^{n}\sigma(z_i)(1-\sigma(z_i))\phi(x_i)\phi(x_i)^T$. Since $\sigma(z_i) \in [0,1]$, the term $\sigma(z_i)(1-\sigma(z_i))$ is ≥ 0. The matrix $\phi(x_i)\phi(x_i)^T$ is positive semi-definite. Therefore, the Hessian H is negative semi-definite, which implies that the empirical log-likelihood $\hat{l}$ is concave. So, doing gradient ascent will converge to a global maximum.

In particular, the sequential formula for gradient ascent will be:

$$\theta_{t+1} = \theta_t + \gamma_t \nabla \hat{l}(\theta_t) = \theta_t + \frac{\gamma_t}{n}\sum_{i=1}^{n}(y_i - h_\theta(x_i))\phi(x_i)$$

Similarly, for stochastic gradient ascent, we can sample u.a.r. one of the n points, and update with $\theta_{t+1} = \theta_t + \gamma_t(y_{i_t} - h_\theta(x_{i_t}))\phi(x_{i_t})$.

4.2 Multiclass Classification: Softmax Regression

We want to generalize the logistic regression to a multiclass setting, where the goal is to predict the label $Y = \{1, 2, \ldots, K\}$. We compute a linear score for each

class $s_k(x) = \phi(x)^T\theta^{(k)}$, then we transform these scores into probabilities using the soft-max function:

$$\hat{p}_k(x) = \frac{e^{s_k(x)}}{\sum_{l=1}^{K} e^{s_l(x)}}.$$

We can interpret the $\hat{p}_k(x)$ as the probability of observing label $Y = k$ when $X = x$. Once we have the $\hat{p}_k(x)$, the most sensible strategy is to output the class with the largest estimated probability.

The empirical log-likelihood to maximize in this multiclass setting thus becomes (using one-hot encoding $y_i^{(k)} = 1$ if $y_i = k$, 0 otherwise):

$$\hat{l}(\Theta) = \frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_i^{(k)} \log(\hat{p}_k(x_i))$$

Note, in this setting, we are optimizing over all the K d-dimensional vectors $\theta^{(k)}$ for $k = 1, \ldots, K$. We use Θ to denote this set of K vectors. We can compute the gradient with respect to the generic $\theta^{(k)}$, to get:

$$\nabla_{\theta^{(k)}}\hat{l}(\Theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i^{(k)} - \hat{p}_k(x_i))\phi(x_i)$$

So, the gradient ascent formula for the k^{th} weight vector becomes

$$\theta_{t+1}^{(k)} = \theta_t^{(k)} + \gamma_t\frac{1}{n}\sum_{i=1}^{n}(y_i^{(k)} - \hat{p}_k(x_i))\phi(x_i)$$

We conclude our study of softmax regression by arguing that the loss function is indeed convex. Before doing so, we prove a preliminary lemma.

Lemma 2.1. The function $f : \mathbb{R}^K \to \mathbb{R}$ (log-sum-exp) defined as follows is convex:

$$f(z_1, \ldots, z_K) = \log\sum_{k=1}^{K} e^{z_k}$$

Proof. We denote with S the sum of the exponential terms, namely $S = \sum_k e^{z_k}$. We start by computing the partial derivatives:

$$\frac{\partial f(z)}{\partial z_i} = \frac{e^{z_i}}{S} = \pi_i$$

$$\frac{\partial^2 f(z)}{\partial z_i^2} = \frac{e^{z_i}S - e^{z_i}e^{z_i}}{S^2} = \frac{e^{z_i}}{S}(1 - \frac{e^{z_i}}{S}) = \pi_i(1 - \pi_i)$$

$$\frac{\partial^2 f(z)}{\partial z_i \partial z_j} = \frac{-e^{z_i} e^{z_j}}{S^2} = -\pi_i \pi_j \quad (i \neq j)$$

We can write the Hessian matrix of f in a convenient way: Denote with π the K-dimensional vector such $\pi_k = e^{z_k}/S$. We have the following:

$$H_{ij} = \frac{\partial^2 f(z)}{\partial z_i \partial z_j} \implies H = \text{diag}(\pi) - \pi\pi^T$$

This matrix H is the covariance matrix of a multinomial distribution with probabilities π, and is known to be positive semi-definite. Therefore, $f(z)$ is convex. □

Theorem 2.2. The function $\hat{l}(\Theta)$ is concave.

Proof. We rewrite $\hat{l}$ using $s_k(x_i) = \phi(x_i)^T \theta^{(k)}$:

$$\begin{aligned}
\hat{l}(\Theta) &= \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_i^{(k)} \log \left(\frac{e^{s_k(x_i)}}{\sum_l e^{s_l(x_i)}} \right) \\
&= \frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_i^{(k)} \left(s_k(x_i) - \log \sum_l e^{s_l(x_i)} \right) \\
&= \frac{1}{n} \sum_{i=1}^{n} \left[\left(\sum_{k=1}^{K} y_i^{(k)} s_k(x_i) \right) - \left(\sum_{k=1}^{K} y_i^{(k)} \right) \log \sum_l e^{s_l(x_i)} \right] \\
&= \frac{1}{n} \sum_{i=1}^{n} \left[\left(\sum_{k=1}^{K} y_i^{(k)} \phi(x_i)^T \theta^{(k)} \right) - \log \sum_l e^{s_l(x_i)} \right] \quad (\text{Since } \sum_k y_i^{(k)} = 1)
\end{aligned}$$

The first term, $\sum_k y_i^{(k)} \phi(x_i)^T \theta^{(k)}$, is a linear function of all the parameters in Θ, therefore it is both concave and convex. The second term is $-f(s(x_i))$, where f is the log-sum-exp function from Lemma 2.1, and $s(x_i)$ is the vector of scores, which is a linear map from Θ. Since f is convex (by Lemma 2.1) and composition with a linear map preserves convexity, $f(s(x_i))$ is convex in Θ. Therefore, $-f(s(x_i))$ is concave in Θ. All in all, $\hat{l}(\Theta)$ is concave as it is the sum of two concave functions (a linear function and a concave function). □

5 Week 5: Perceptron and SVM

5.1 The Perceptron Algorithm

We introduce one of the seminal machine learning algorithms: the Perceptron. It performs binary classification via separating hyperplanes, under the assumption that the underlying dataset is separable, i.e., there exists a hyperplane that perfectly separates the points corresponding to the two labels.

Formally, we are given n points $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$, with $x_i \in \mathcal{X} = \mathbb{R}^d$, $y_i \in \mathcal{Y} = \{-1, 1\}$ and we want to find an hyperplane H_θ that separates them.

Recall, a generic hyperplane H_θ is given by all the points orthogonal to a non-zero vector θ: $H_\theta = \{x \in \mathcal{X} \mid x^T\theta = 0\}$. Given a vector $\theta \in \mathbb{R}^d$ and a point $x \in \mathbb{R}^d$. The label y predicted for x by θ is then $\text{sign}(x^T\theta)$. (e.g., $+1$ if $x^T\theta \geq 0$, and -1 if $x^T\theta < 0$).

Definition 1.1. Given a vector θ, and a point x with label $y \in \{-1, 1\}$, we say that H_θ makes a classification error on x if $y \cdot x^T\theta \leq 0$. If we have that $y \cdot x^T\theta < 1$, then we say that there is a margin error.

Remark 1.2. As we have already done for linear regression, we are considering homogeneous hyperplanes, i.e., those that contain the origin. This is, again, without loss of generality, as we can always add an extra dimension and set the corresponding coordinate to 1 for all the points in the dataset.

The Perceptron algorithm, described in the pseudocode, works as follows. It maintains a candidate hyperplane identified by vector θ_t, then considers the points in the dataset one after the other (either cyclically, or uniformly at random as reported here). Every time that the current solution θ_t makes a margin error on the point (x_{i_t}, y_{i_t}) considered, it modifies θ_t by adding $y_{i_t}x_{i_t}$.

Given a point x and an hyperplane H_θ, the distance of x to the hyperplane is:

$$\text{dist}(x, H_\theta) = \min_{z \in H_\theta} \|x - z\| = \frac{|x^T\theta|}{\|\theta\|}.$$

Definition 1.3. Given $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ and an hyperplane H_θ that correctly classifies all the points, the margin of H_θ, denoted with $\gamma(\theta)$ is defined as the distance of H_θ from its closest point:

$$\gamma(\theta) = \min_{i=1,\ldots,n} \text{dist}(x_i, H_\theta).$$

The largest possible margin achievable by a hyperplane that classifies the points correctly is denoted with $\gamma^\star$.

Algorithm 1 The Perceptron Algorithm

Require: Training data $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$
Require: Maximum number of iterations N
Initialize $\theta_0 \leftarrow 0 \in \mathbb{R}^d$
for $t = 1, 2, \ldots, N$ **do**
    Let i_t be an index drawn uniformly at random in $1, \ldots, n$
    if $y_{i_t} \cdot x_{i_t}^T \theta_t < 1$ **then** ▷ Margin Error
        $\theta_{t+1} \leftarrow \theta_t + y_{i_t} x_{i_t}$ ▷ θ_t is adjusted
    else
        $\theta_{t+1} \leftarrow \theta_t$ ▷ θ_t is not updated
    end if
end for
return θ_N

We are ready to provide an upper bound on the number of margin mistakes that the Perceptron Algorithm makes. We say that the algorithm converges if it finds a hyperplane that correctly classifies all the points and makes no margin error. Finally, we denote with D the diameter of the problem, i.e. $D = \max_i \|x_i\|$, and γ^* the largest possible margin.

Theorem 1.4. (Novikoff) The Perceptron Algorithm makes at most $M = \frac{2+D^2}{(\gamma^*)^2}$ margin mistakes, if the points are linearly separable with margin γ^*.

Proof. Consider a generic time step t where θ_t makes a margin error on (x_{i_t}, y_{i_t}), we have:

$$\|\theta_{t+1}\|^2 = \|\theta_t + y_{i_t} x_{i_t}\|^2 = \|\theta_t\|^2 + 2 y_{i_t} x_{i_t}^T \theta_t + \|x_{i_t}\|^2 \leq \|\theta_t\|^2 + 2 + D^2$$

Denote with m_t the number of margin errors made up to time t included. If we repeatedly apply the above inequality, we get:

$$\|\theta_{t+1}\|^2 \leq m_t(2 + D^2) + \|\theta_0\|^2 = m_t(2 + D^2).$$

Denote now with θ^* a unit norm vector that identifies the hyperplane with the largest margin, so that $\gamma^* = \min_i y_i x_i^T \theta^*$.

$$(\theta^*)^T \theta_{t+1} = \sum_{k=1}^{m_t} (\theta^*)^T (y_{i_k} x_{i_k}) = \sum_{k=1}^{m_t} y_{i_k} (\theta^*)^T x_{i_k} \geq \sum_{k=1}^{m_t} \gamma^* = m_t \gamma^*$$

Now, by Cauchy-Schwarz: $(\theta^*)^T \theta_{t+1} \leq \|\theta^*\| \cdot \|\theta_{t+1}\| = \|\theta_{t+1}\|$ (since $\|\theta^*\| = 1$). So, $m_t \gamma^* \leq \|\theta_{t+1}\|$. Squaring both sides: $(m_t \gamma^*)^2 \leq \|\theta_{t+1}\|^2$. Combining with the upper bound:

$$(m_t \gamma^*)^2 \leq \|\theta_{t+1}\|^2 \leq m_t(2 + D^2)$$

$$m_t^2(\gamma^*)^2 \leq m_t(2+D^2) \implies m_t \leq \frac{2+D^2}{(\gamma^*)^2}$$

This concludes the proof. □

5.2 Support Vector Machine

For the perceptron algorithm to work, we need to assume separability. Another way of looking at this classification task is to solve the following hard-margin problem:

$$\min_{\theta} \frac{1}{2}\|\theta\|^2 \quad \text{subject to} \quad y_i(x_i^T\theta) \geq 1, \quad \forall i$$

There are two reasons to minimize $\|\theta\|$. First, it is always good to choose the "simplest" solution. Second, there is a strong relation between $\|\theta\|$ and the margin $\gamma(\theta)$.

Proposition 2.1. Consider any $\hat{\theta}$ that solves the hard-margin problem, then the margin of $H_{\hat{\theta}}$ is optimal, $\gamma(\hat{\theta}) = \gamma^*$, and $\gamma^* = 1/\|\hat{\theta}\|$.

Proof. Consider any θ that satisfies the constraints $y_i x_i^T\theta \geq 1$. The margin is

$$\gamma(\theta) = \min_i \frac{|x_i^T\theta|}{\|\theta\|} = \min_i \frac{y_i x_i^T\theta}{\|\theta\|} \geq \frac{1}{\|\theta\|}.$$

Let θ^* be the optimal vector. We can scale it so that $\min_i y_i x_i^T\theta^* = 1$. For this θ^*, we have $\gamma(\theta^*) = \min_i \frac{y_i x_i^T\theta^*}{\|\theta^*\|} = \frac{1}{\|\theta^*\|}$. Since $\hat{\theta}$ is the solution that minimizes $\|\theta\|$ while satisfying the constraints, we have $\|\hat{\theta}\| \leq \|\theta^*\|$. Thus, $\gamma(\hat{\theta}) \geq \frac{1}{\|\hat{\theta}\|} \geq \frac{1}{\|\theta^*\|} = \gamma(\theta^*) = \gamma^*$. Since γ^* is the largest possible margin, we must have $\gamma(\hat{\theta}) = \gamma^*$. □

What can we say when the underlying dataset is not linearly separable? We introduce some slackness. For any point i, we associate a slackness variable $\xi_i \geq 0$ and we relax the margin error: $y_i x_i^T\theta \geq 1 - \xi_i$. Problem (3) then becomes (Soft-Margin SVM):

$$\min_{\theta,\xi} \frac{1}{2}\|\theta\|^2 + C\sum_{i=1}^{n}\xi_i$$
$$\text{subject to} \quad y_i(x_i^T\theta) \geq 1 - \xi_i, \quad \forall i$$
$$\xi_i \geq 0, \quad \forall i$$

Note, C is a hyperparameter that tunes the relative importance of the norm term and of the margin error.

We can further simplify the optimization problem, by noting that, for any fixed vector θ, the best choice of ξ_i is $\xi_i = \max\{0, 1 - y_i\theta^T x_i\} = (1 - y_i\theta^T x_i)_+$, where $(\cdot)_+$

denotes the positive part. Therefore, we can rewrite the equivalent (unconstrained) optimization problem:

$$\min_{\theta} \frac{1}{2}\|\theta\|^2 + C\sum_{i=1}^{n}(1 - y_i\theta^T x_i)_+$$

The function $h(z) = (1-z)_+$ is called the hinge function, and is convex. Therefore, the objective function is convex (sum of two convex functions), and we can safely use gradient or stochastic gradient descent. There is a small caveat: there are some points where the gradient may not be well defined (at $z = 1$). In those points, we use a sub-gradient.

Definition 2.2. Let $F : \mathbb{R}^d \to \mathbb{R}$ be a convex function, and $x \in \mathbb{R}^d$. We say that a vector $g \in \mathbb{R}^d$ is a sub-gradient in x, if the following inequality holds for any $y \in \mathbb{R}^d$:

$$F(y) \geq F(x) + g^T(y - x)$$

Let $L(\theta)$ be our objective function. $L(\theta) = \frac{1}{2}\|\theta\|^2 + C\sum_i L_i(\theta)$ where $L_i(\theta) = (1 - y_i\theta^T x_i)_+$. The sub-gradient of $L_i(\theta)$ is:

$$\partial L_i(\theta) = \begin{cases} \{-Cy_i x_i\} & \text{if } y_i\theta^T x_i < 1 \\ \{0\} & \text{if } y_i\theta^T x_i > 1 \\ \{\alpha(-Cy_i x_i) \mid \alpha \in [0,1]\} & \text{if } y_i\theta^T x_i = 1 \end{cases}$$

A valid sub-gradient $g(\theta)$ for $L(\theta)$ can be constructed by taking one sub-gradient for each term:

$$g(\theta) = \theta + \sum_i g_i \quad \text{where } g_i \in \partial(C \cdot L_i(\theta))$$

A common choice is:

$$g(\theta) = \theta - C\sum_i y_i x_i \mathbb{1}_{\{y_i\theta^T x_i \leq 1\}}$$

At this point, it is instructive to look at the pseudocode for the stochastic (sub-)gradient version:

A natural choice for the learning rate is $\eta_t = \frac{1}{Ct}$. The stochastic sub-gradient descent algorithm for SVM is extremely similar to the Perceptron Algorithm, even though they are designed for slightly different problems. Moreover, this model is called Support Vector Machine (SVM), because the decision boundary only depends on the points considered, where margin errors have been made (the "support vectors").

Algorithm 2 Stochastic Gradient Descent for SVM

Require: Training data $\{(x_i, y_i)\}_{i=1}^n$, $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$
Require: Maximum iterations N, learning rates η_t, regularizer C
Initialize $\theta_0 \leftarrow 0 \in \mathbb{R}^d$
for $t = 1, 2, \ldots, N$ **do**
    Let i_t be an index drawn uniformly at random in $1, \ldots, n$
    if $y_{i_t} \cdot x_{i_t}^T \theta_t < 1$ **then** ▷ Margin Error
        $\theta_{t+1} \leftarrow \theta_t - \eta_t(\theta_t - C y_{i_t} x_{i_t})$ ▷ Update with margin-violating point
    else
        $\theta_{t+1} \leftarrow \theta_t(1 - \eta_t)$ ▷ Update with only regularization term
    end if
end for
return θ_N

5.3 Kernel Trick

Now, imagine that we want to go beyond linear separators. A possibility may be to map the points to a feature space via a feature map $\psi : \mathbb{R}^d \to \mathbb{R}^{d'}$. We do not want to find an algorithm that depends on the feature dimension d', which may be infinite or very large.

Since we want a definition that also covers infinitely dimensional vector space, we state the formal definition of scalar products (or inner products).

Definition 2.3. Given a vector space V, a scalar product $\langle \cdot, \cdot \rangle : V^2 \to \mathbb{R}$ satisfies the following properties:

- Linearity: for any vectors $v, w, z \in V$ and scalar $\alpha \in \mathbb{R}$, it holds $\langle \alpha v, z \rangle = \alpha \langle v, z \rangle$ and $\langle v + w, z \rangle = \langle v, z \rangle + \langle w, z \rangle$.
- Symmetry: for any vectors $v, w \in V$ it holds $\langle v, w \rangle = \langle w, v \rangle$.
- Positivity: for any vector $v \in V$, $\langle v, v \rangle \geq 0$, and the equality is only reached for $v = 0$.

The last ingredient we need is the notion of Hilbert space: essentially, an Hilbert Space is a vector space equipped with a scalar product.

There are two crucial observations underlying the so-called Kernel Trick:

- The vector θ_t maintained by SVM (and Perceptron) is always a linear combination of vectors of the form $\psi(x_1), \ldots, \psi(x_n)$. (This is known as the Representer Theorem). Therefore all the optimization procedure lives in this n-dimensional subspace. We only need to maintain the n coordinates

$\alpha_1^t, \ldots, \alpha_n^t$ in memory, that identify $\theta_t = \sum_{i=1}^n \alpha_i^t \psi(x_i)$.

- We actually do not need to know exactly θ_t, but only to be able to compute scalar products of the type $\langle \psi(x_j), \theta_t \rangle$ (for prediction) and update the α_i's.

$$\langle \psi(x_j), \theta_t \rangle = \left\langle \psi(x_j), \sum_{i=1}^n \alpha_i^t \psi(x_i) \right\rangle = \sum_{i=1}^n \alpha_i^t \langle \psi(x_j), \psi(x_i) \rangle.$$

Therefore, at the end of the day, we only need to maintain in memory the n coordinates of θ_t (the α_i's), and be able to compute the so-called kernel matrix K whose generic element is $K(x_i, x_j) = \langle \psi(x_i), \psi(x_j) \rangle$. This $K(x_i, x_j)$ is the "kernel function".

Example 2.4 (Polynomial Kernel). Imagine $d = 1$, and we consider the feature maps $\psi(x) = (1, x, x^2, \ldots, x^D)$. The target space is $\mathbb{R}^{D+1}$, and the scalar product becomes $\langle \psi(x), \psi(z) \rangle = \sum_{i=0}^D (xz)^i$. A more general polynomial kernel is $K(x, z) = (x^T z + c)^D$.

Example 2.5 (Gaussian Kernel). The dimension of the input space is d, we can map the points to an infinite dimensional vector space whose scalar product is computed as follows: $K(x, z) = \langle \psi(x), \psi(z) \rangle = e^{-\|x-z\|^2/(2\sigma^2)}$. This type of Kernel is called Gaussian or Radial Basis Function (RBF) Kernel.

6 Week 6: Statistical Learning and Tree Predictors

6.1 Generalization Bounds

Our goal is to provide a mathematical framework for the overfitting phenomenon. From previous weeks, we have the high-level intuition that "more complex" models require more data to achieve the same performance on the test data. This perspective is provided by so-called generalization bounds, which quantify the number of samples required to ensure that the expected risk of all the classifiers is well estimated by the corresponding empirical risk. In these notes we focus on binary classification (labels $\mathcal{Y} = \{0, 1\}$ or $\{-1, 1\}$), but similar results hold for multi-class classification and regression.

Assume that we have access to n i.i.d. samples from a distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, and we consider a hypothesis class $\mathcal{H}$ of classifiers. For each $h \in \mathcal{H}$ we use the empirical risk $\hat{R}$ to estimate the expected risk R. Concretely:

$$R(h) = \mathbb{E}_{(X,Y)\sim\mathcal{D}}\big[l(h(X), Y)\big], \qquad \hat{R}(h) = \frac{1}{n}\sum_{i=1}^{n} l(h(x_i), y_i).$$

We would like to argue that for all classifiers $h \in \mathcal{H}$ it holds that $\hat{R}(h) \approx R(h)$.

6.1.1 Mathematical preliminaries

We recall two fundamental probabilistic tools: the union bound and Hoeffding/Chernoff inequality.

Theorem (Union bound). Given events $A_1, \ldots, A_k$, then

$$\mathbb{P}\big(A_1 \cup \cdots \cup A_k\big) \leq \sum_{i=1}^{k} \mathbb{P}(A_i).$$

Theorem (Hoeffding / Chernoff bound). Let $Z_1, \ldots, Z_n$ be independent Bernoulli random variables with $\mathbb{E}[Z_i] = \mu$. Let $\hat{Z} = \frac{1}{n}\sum_{i=1}^{n} Z_i$. Then for any $\varepsilon > 0$,

$$\mathbb{P}\big(|\hat{Z} - \mu| > \varepsilon\big) \leq 2e^{-2\varepsilon^2 n}.$$

6.1.2 Finite families of classifiers

When the hypothesis class $\mathcal{H}$ is finite we obtain a simple uniform convergence bound.

Theorem. Let $\varepsilon \in (0,1)$. For a finite family $\mathcal{H}$, with n i.i.d. samples,

$$\mathbb{P}\Big(\sup_{h\in\mathcal{H}} |R(h) - \hat{R}(h)| > \varepsilon\Big) \le 2|\mathcal{H}|e^{-2\varepsilon^2 n}.$$

Proof. For each $h \in \mathcal{H}$ define the event $E_h = \{|R(h) - \hat{R}(h)| \le \varepsilon\}$. Then

$$\mathbb{P}\Big(\exists h \in \mathcal{H} :\ |R(h) - \hat{R}(h)| > \varepsilon\Big) = \mathbb{P}\Big(\bigcup_{h\in\mathcal{H}} E_h^c\Big) \le \sum_{h\in\mathcal{H}} \mathbb{P}(E_h^c) \le |\mathcal{H}| \cdot 2e^{-2\varepsilon^2 n},$$

where the last inequality follows from Hoeffding's bound applied to each fixed h. □

Corollary. Fix $\varepsilon \in (0,1)$ and $\delta \in (0,1)$. If

$$n \ge \frac{1}{2\varepsilon^2} \ln \frac{2|\mathcal{H}|}{\delta},$$

then with probability at least $1-\delta$ (over the draw of the training sample) we have $\sup_{h\in\mathcal{H}} |R(h) - \hat{R}(h)| \le \varepsilon$.

Important. The randomness is with respect to the training sample: the result states that with probability at least $1-\delta$ over the random draw of the n i.i.d. training points, empirical risks approximate true risks uniformly over $\mathcal{H}$.

6.1.3 Infinite families: VC dimension

Most interesting hypothesis classes are infinite (e.g., linear classifiers). The Vapnik–Chervonenkis (VC) dimension is a combinatorial measure of complexity that controls uniform convergence.

Definition (Shattering). Given a finite set $S = \{x_1, \dots, x_m\} \subset \mathcal{X}$, we say that $\mathcal{H}$ *shatters* S if for every labeling $(y_1, \dots, y_m) \in \{0,1\}^m$ there exists $h \in \mathcal{H}$ with $h(x_i) = y_i$ for all i.

Definition (VC dimension). The VC dimension of $\mathcal{H}$, denoted $\mathrm{VC}(\mathcal{H})$, is the size of the largest finite set S that $\mathcal{H}$ shatters. If $\mathcal{H}$ can shatter arbitrarily large finite sets, $\mathrm{VC}(\mathcal{H}) = \infty$.

The following (standard) uniform convergence theorem links VC dimension and sample complexity (see e.g. Shalev-Shwartz & Ben-David).

Theorem. Let $\mathcal{H}$ have VC-dimension d. Fix $\varepsilon, \delta \in (0,1)$. There exist absolute constants C_1, C_2 such that if

$$n \ge C_1 \frac{d \log(C_2/\varepsilon) + \log(1/\delta)}{\varepsilon^2},$$

then with probability at least $1-\delta$,

$$\sup_{h\in\mathcal{H}} |R(h) - \hat{R}(h)| \le \varepsilon.$$

Note. Informally, sample complexity scales roughly like $\frac{d}{\varepsilon^2}$ up to logarithmic factors; in particular it grows linearly with the VC dimension d.

Example.

- The VC dimension of intervals on the real line is 2.
- The VC dimension of axis-aligned rectangles in $\mathbb{R}^2$ is 4.
- The VC dimension of triangles in $\mathbb{R}^2$ is 7.
- For a finite class $\mathcal{H}$, $\mathrm{VC}(\mathcal{H}) = O(\log|\mathcal{H}|)$ (more precisely $\le \lfloor \log_2 |\mathcal{H}| \rfloor$ under binary outputs).

6.1.4 Risk decomposition

Let $h^\star$ denote the Bayes optimal classifier and let $\mathcal{H}$ be a hypothesis class. For any $h \in \mathcal{H}$,

$$R(h) = R(h^\star) + \big(\inf_{h\in\mathcal{H}} R(h) - R(h^\star)\big) + \big(R(h) - \inf_{h\in\mathcal{H}} R(h)\big).$$

The three terms are respectively: Bayes error (irreducible), approximation (bias) error due to restricting to $\mathcal{H}$, and estimation error within $\mathcal{H}$. Uniform convergence bounds control the third term: if $d = \mathrm{VC}(\mathcal{H})$, then for suitable n,

$$\sup_{h\in\mathcal{H}} |R(h) - \hat{R}(h)| = O\Big(\sqrt{\frac{d\log n + \log(1/\delta)}{n}}\Big),$$

so choosing the empirical risk minimizer (or near minimizer) yields estimation error of that order.

Important. There is a bias–variance (approximation–estimation) tradeoff controlled by the complexity of $\mathcal{H}$: larger $\mathcal{H}$ reduces approximation error but increases estimation error (needs more data to control).

6.2 Tree Predictors

Assume the input space decomposes as $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d$. A *tree predictor* is represented by a rooted binary tree whose internal nodes perform tests (on coordinates of x) and leaves carry labels. To label x, follow the root-to-leaf path

determined by the outcomes of the tests; the label at the reached leaf is the prediction.

Let $\mathcal{X} = [0,1]^2$. A simple tree: test $x^{(1)} \geq 1/2$ to split left/right; then on the right branch test $x^{(2)} \geq 1/3$ to decide label; on the left branch test $x^{(2)} \leq 2/3$ to decide label.

In what follows we consider binary input $\mathcal{X} = \{0,1\}^d$ and binary labels $\{-1,+1\}$ with 0–1 loss for simplicity (extension to more general settings is straightforward). Internal nodes are binary tests and each internal node has two children.

6.2.1 Training a tree predictor

Given a tree T and training set $\{(x_i, y_i)\}_{i=1}^n$, each training point ends up in a leaf ℓ. Let N_ℓ be the number of points in leaf ℓ, and N_ℓ^+, N_ℓ^- the counts of $+1$ and -1 labels. Label leaf ℓ by the majority label. The empirical error contribution of leaf ℓ is $\min\{N_\ell^+, N_\ell^-\}$. Thus the empirical error of tree T is

$$\hat{R}(T) = \frac{1}{n} \sum_{\ell \text{ leaf}} \min\{N_\ell^+, N_\ell^-\} = \frac{1}{n} \sum_{\ell \text{ leaf}} N_\ell \psi\Big(\frac{N_\ell^+}{N_\ell}\Big),$$

where $\psi(z) = \min\{z, 1-z\}$.

Theorem. For any training set with distinct inputs (i.e., $x_i \neq x_j$ for $i \neq j$), there exists a tree predictor that correctly classifies all training points.

Proof (Sketch). Grow a tree that isolates each training point into its own leaf (by splitting coordinates or otherwise). Label each leaf according to the true label of the single point inside it. This yields zero training error. □

This shows that unconstrained trees can overfit. A common approach is to restrict tree size (number of nodes N) and seek the best tree with at most N nodes. A standard greedy algorithm (CART-style) iteratively splits leaves to reduce empirical impurity until a node budget is reached.

When replacing a leaf ℓ by two leaves ℓ_L, ℓ_R, the empirical error cannot increase if the impurity function is concave; for $\psi(z) = \min\{z, 1-z\}$ (concave on $[0,1]$) we have

$$N_\ell \psi\Big(\frac{N_\ell^+}{N_\ell}\Big) \geq N_{\ell_L} \psi\Big(\frac{N_{\ell_L}^+}{N_{\ell_L}}\Big) + N_{\ell_R} \psi\Big(\frac{N_{\ell_R}^+}{N_{\ell_R}}\Big).$$

In practice surrogate impurity functions are commonly used:

$$\text{Gini: } \psi_G(z) = 2z(1-z), \qquad \text{(scaled) entropy: } \psi_e(z) = -\frac{z}{2}\log z - \frac{1-z}{2}\log(1-z),$$

which are smooth and convenient for optimization.

6.2.2 VC dimension of tree predictors

The class of all trees (unrestricted depth) has infinite VC dimension: for any finite set of points we can build a deep tree that maps each point to a distinct leaf and thus realizes any labeling. However, trees with a bounded number of nodes N form a finite (though large) class and therefore have finite VC dimension.

Proposition. Let $\mathcal{T}_N$ be the family of all binary tree predictors with at most N nodes, on inputs in $\{0,1\}^d$. Then

$$|\mathcal{T}_N| \leq (8d)^N.$$

Consequently the sample complexity to learn $\mathcal{T}_N$ scales linearly in N (up to logarithmic factors).

Proof (Sketch). Upper-bound the number of structures, tests and leaf labels. The number of (unlabeled) rooted binary trees with N nodes is at most 4^N (Catalan-type growth). Each internal node can choose among at most d possible coordinate tests, giving a factor d^N, and each leaf can be labeled in two ways, giving a factor 2^N. Multiplying yields a bound $\leq 4^N \cdot d^N \cdot 2^N = (8d)^N$. □

6.2.3 Random Forests

Random Forests combine many (short) trees trained on different bootstrap samples and/or random feature subsets. Each tree votes and the forest prediction is the majority (possibly weighted) vote. This ensembling reduces variance and typically improves generalization over a single tree.